PYTHON ACTIVITY-BASED LEARNING

10 Simple Case Studies with Complete Documentation

Prepared according to the case-study list and required deliverables in the college circular.

Student Name:AngatI.VarshiniSriLakshmi

Roll Number: 26001373

Branch / Section: CSE – Data Science

.

Suggested GitHub repository name: Python-Activity-Based-Learning

Case Study 1: University Scholarship Eligibility

1. Problem Statement

WriteaPythonprogramtodecidewhetherastudentiseligibleforascholarshipusingaggregatemarks,attendance, and family income.

2. Objectives

• Use variables and input values.

• Apply multiple conditions using if-else.

• Display a clear eligibility result.

3. Assumptions

• Marks are entered as a percentage from 0 to 100.

• Attendance is entered as a percentage from 0 to 100.

• Scholarship rules: marks must be at least 80, attendance at least 75, and family income no more than ■250000.

4. Algorithm

1. Start.

2. Read marks, attendance, and annual family income.

3. Check that marks and attendance are within 0–100 and income is non-negative.

4. If any input is invalid, display an error.

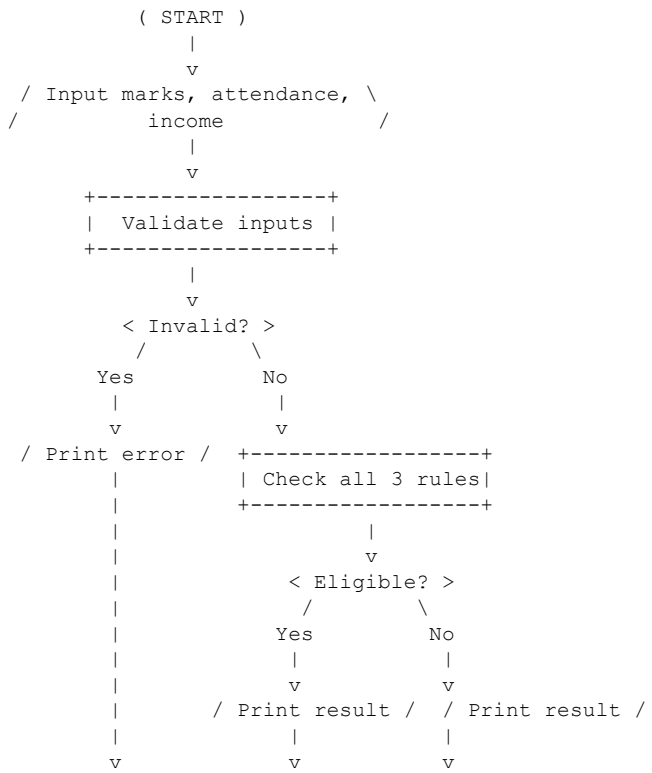
5. Otherwise, check all three scholarship conditions.

6. Display eligible or not eligible.

7. Stop.

5. Flowchart

```
      START →Input marks, attendance, income →Validate inputs →Invalid? ■Yes→Print error →END
                                             ■No→Check all 3 rules →Eligible? →Print result →END
```



```
          ( END ) <------ + ---------+
```

6. Pseudocode

```
BEGIN
  INPUTmarks,attendance,income
  IF marks < 0 OR marks > 100 OR attendance < 0 OR attendance > 100 OR income <0
      PRINT "Invalid input"
  ELSE IF marks >= 80 AND attendance >= 75 AND income <= 250000
      PRINT "Eligible for scholarship"
  ELSE
      PRINT "Not eligible for scholarship"
  END IF
END
```

7. Python Source Code

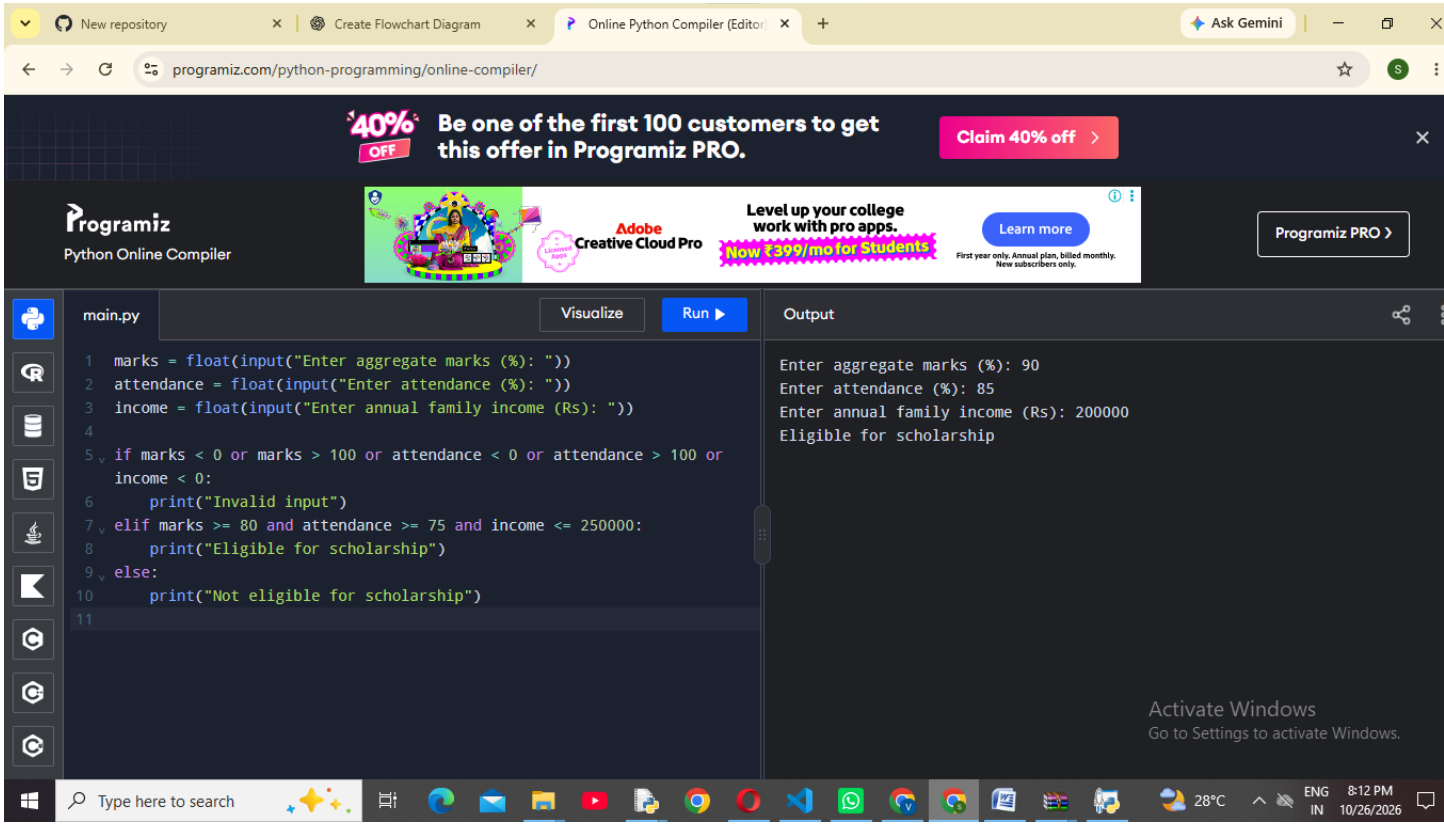
```
marks = float(input("Enter aggregate marks (%): "))
attendance=float(input("Enterattendance(%):"))
income=float(input("Enterannualfamilyincome(Rs):"))

if marks < 0 or marks > 100 or attendance < 0 or attendance > 100 or income <0:
    print("Invalid input")
elif marks >= 80 and attendance >= 75 and income <= 250000:
    print("Eligible for scholarship")
else:
    print("Noteligibleforscholarship")
```

8. Test Cases and Sample Output

Test input	Expected output
90, 85, 200000	Eligible for scholarship
78, 90, 200000	Not eligible for scholarship
90, 70, 200000	Not eligible for scholarship

Sample console output :



9. Conclusion

The program uses conditional statements to check academic, attendance ,and in omerquirements and prints the scholarship decision.

Case Study 2: Progressive Electricity Billing

1. Problem Statement

Write a program to calculate a household electricity bill using progressive unit slabs and a fixed charge.

2. Objectives

• Accept electricity units used.

• Calculate charges using if-elif-else.

• Display the total bill.

3. Assumptions

• Units cannot be negative.

• First 100 units cost ■2 per unit; next 100 units cost ■3 per unit; units above 200 cost ■5 per unit.

• A fixed charge of ■50 is added.

4. Algorithm

1. Start.

2. Read units consumed.

3. If units are negative, display an error.

4. For up to 100 units, charge all units at ■2.

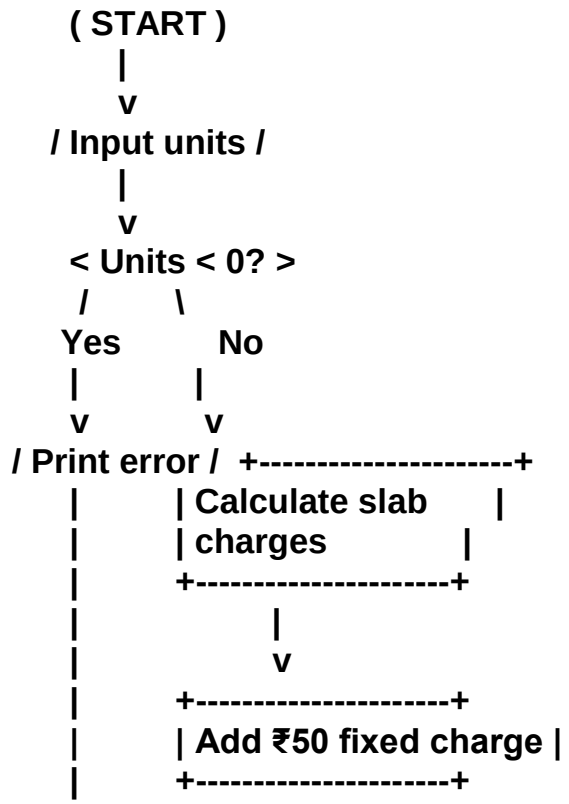
5. For 101–200 units, charge first 100 at ■2 and the rest at ■3.

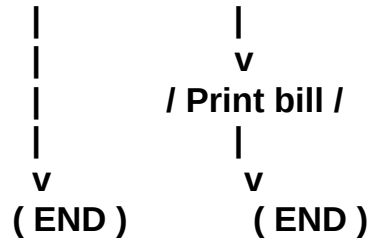
6. Above 200 units, charge the first 100 at ■2, next 100 at ■3, and remaining units at ■5.

7. Add ■50 fixed charge and display bill.

8. Stop.

5. Flowchart





6. Pseudocode

```
BEGIN
  INPUT units
  IFunits<0
      PRINT"Invalidunits"
  ELSE
      IFunits<=100:energy_charge=units*2
      ELSE IF units <= 200: energy_charge = 100*2 + (units-100)*3
      ELSE: energy_charge = 100*2 + 100*3 + (units-200)*5
      bill=energy_charge+50 PRINT
      bill
  ENDIF
END
```

7. Python Source Code

```
units=int(input("Enterunitsconsumed:"))

ifunits<0:
    print("Invalidunits")
else:
    if units <= 100:
        energy_charge=units*2
    elifunits<=200:
        energy_charge = 100 * 2 + (units - 100) * 3
    else:
```

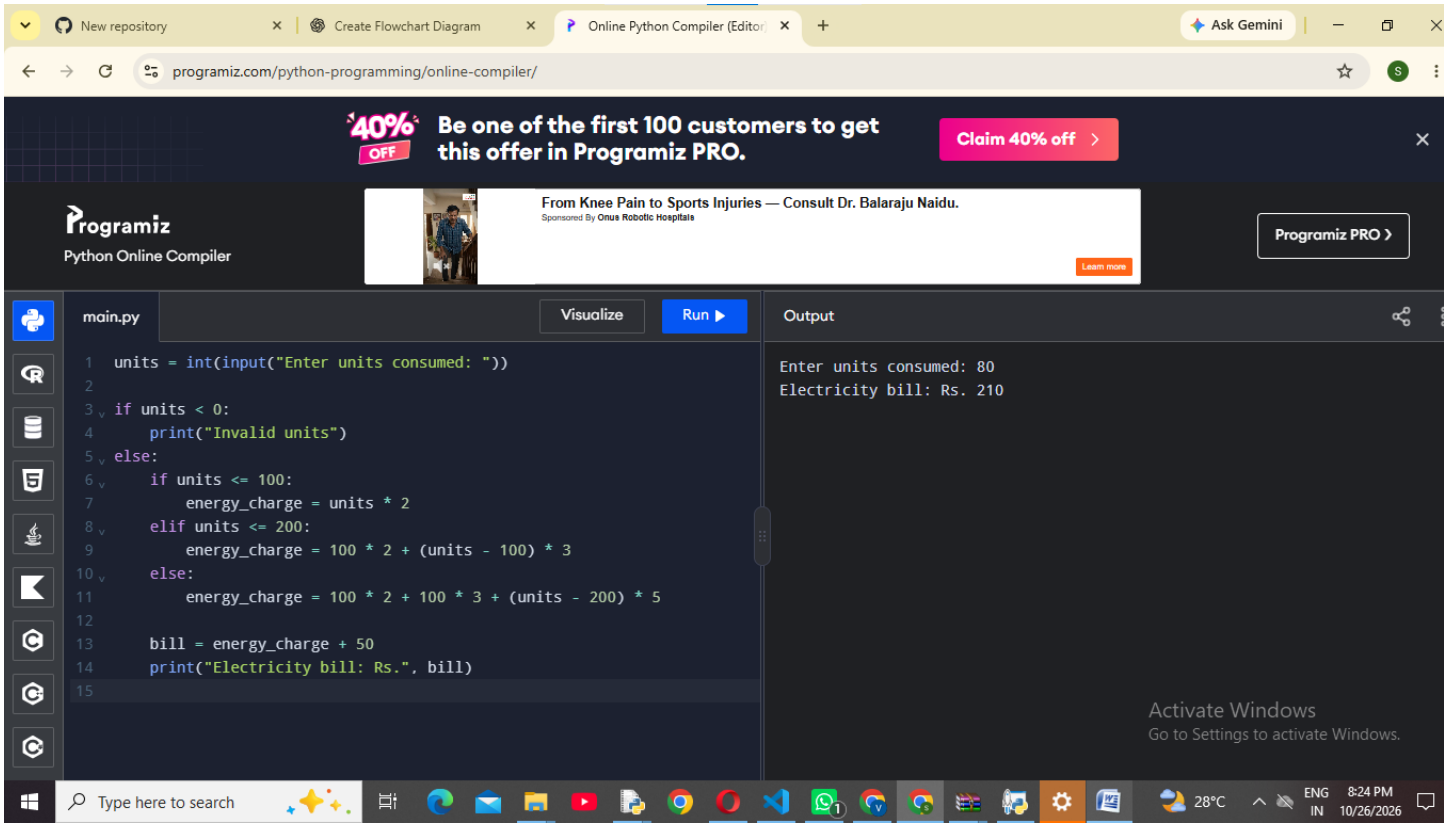
```
    energy_charge=100*2+100*3+(units-200)*5

bill = energy_charge + 50
print("Electricity bill: Rs.", bill)
```

8. Test Cases and Sample Output

Test input	Expected output
80	Electricity bill: Rs. 210
150	Electricity bill: Rs. 400
250	Electricity bill: Rs. 850

Sample console output:



9. Conclusion

The program calculates the bill by applying different rates to each unit slab and adding a fixed charge.

Case Study 3: Examination Result and Pass/Fail

1. Problem Statement

Writeaprogramthatacceptsmarksinthreesubjects,calculatestheaverage,anddecideswhetherthestudent passes every subject.

2. Objectives

• Use input, loops, and lists.

• Check subject-wise pass marks.

• Calculate total and average marks.

3. Assumptions

• Each subject is out of 100.

• A student must score at least 35 in every subject to pass.

• Marks must be between 0 and 100.

4. Algorithm

1. Start.

2. Read marks for three subjects into a list.

3. Validate each mark.

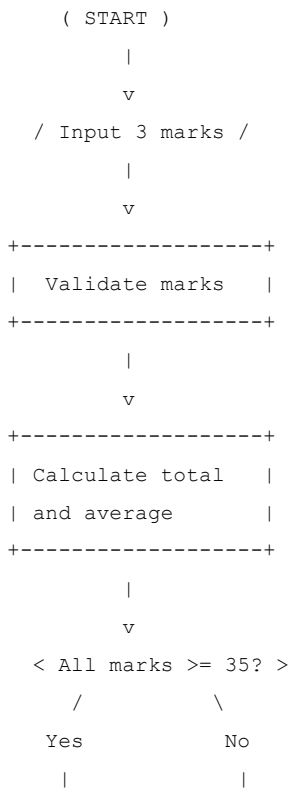
4. Calculate total and average.

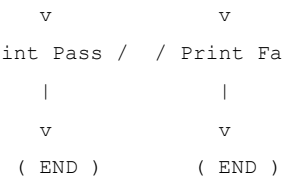
5. Check whether every subject mark is at least 35.

6. Display total, average, and result.

7. Stop.

5. Flowchart





6. Pseudocode

```
BEGIN
  marks=emptylist REPEAT
  3 times
      INPUTmark
      ADDmarktomarks END
  REPEAT
  IF any mark is outside 0..100: PRINT "Invalid
  marks"ELSE
      total=sumofmarks
      average = total / 3
      IF every mark >= 35: result = "PASS"
      ELSE: result = "FAIL"
      PRINT total, average, result
  END IF
END
```

7. Python Source Code

```
marks=[]

forsubjectinrange(1,4):
    mark = int(input("Enter marks for subject " + str(subject) + ": "))
    marks.append(mark)

valid=True
formarkinmarks:
    ifmark<0ormark>100:
```

```
        valid=False

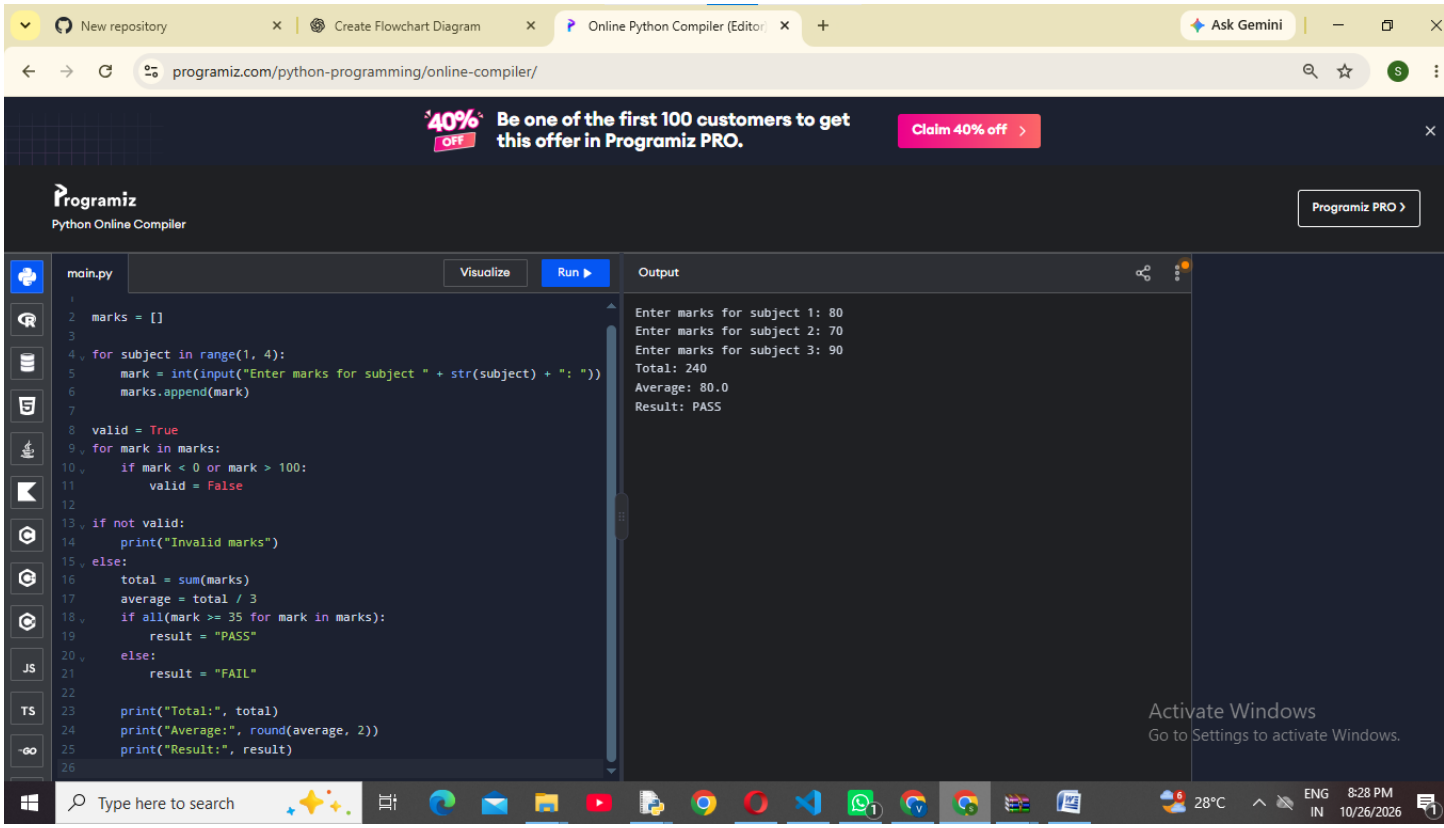
ifnotvalid:
    print("Invalidmarks")
else:
    total = sum(marks)
    average=total/3
    if all(mark >= 35 for mark in marks):
        result = "PASS"
    else:
        result="FAIL"

    print("Total:", total)
    print("Average:", round(average, 2))
    print("Result:", result)
```

8. Test Cases and Sample Output

Test input	Expected output
80, 70, 90	Total: 240; Average: 80.0; Result: PASS
80, 30, 90	Total: 200; Average: 66.67; Result: FAIL
35, 35, 35	Total: 105; Average: 35.0; Result: PASS

Sample console output :



9. Conclusion

The program uses a list and loop to process subject marks and determines the result using subject-wise pass criteria.

Case Study 4: Warehouse Inventory Reconciliation

1. Problem Statement

Write a program to calculate expected closing stock and compare it with the physically counted stock.

2. Objectives

- Use basic arithmetic and conditions.
- Calculate expected stock from opening stock, received stock, and dispatched stock.
- Identify stock discrepancies.

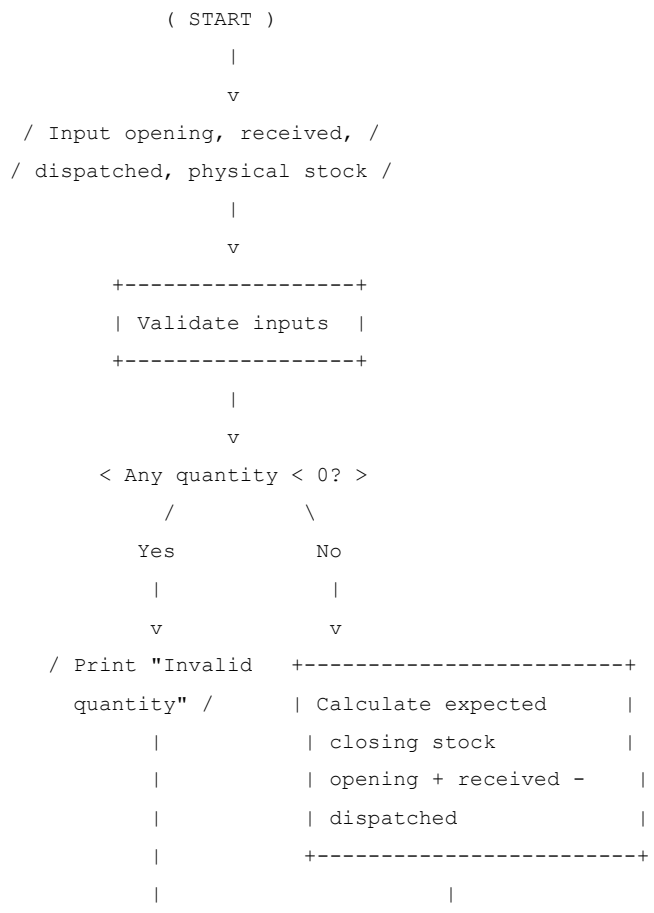
3. Assumptions

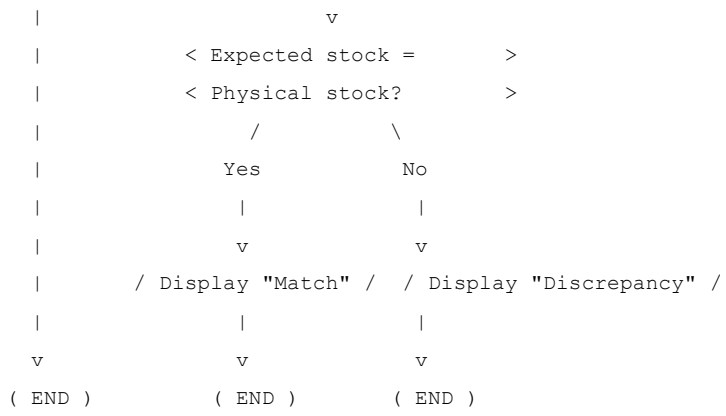
- All quantities must be non-negative integers.
- Expected closing stock = opening stock + received stock −dispatched stock.

4. Algorithm

1. Start.
2. Read opening, received, dispatched, and physical closing quantities.
3. Validate that all quantities are non-negative.
4. Calculate expected closing stock.
5. Compare expected stock with physical stock.
6. Display matching stock or discrepancy.
7. Stop.

5. Flowchart





6. Pseudocode

```
BEGIN
  INPUT opening, received, dispatched, physical
  IF any quantity < 0
      PRINT"Invalidquantity"
  ELSE
      expected = opening + received - dispatched
      difference = physical - expected
      PRINTexpected
      IF difference == 0: PRINT "Stock matches"
      ELSE: PRINT difference
  ENDIF
END
```

7. Python Source Code

```
opening = int(input("Opening stock: "))
received = int(input("Quantity received: "))
dispatched = int(input("Quantity dispatched: "))
physical=int(input("Physicalclosingstock:"))

if opening < 0 or received < 0 or dispatched < 0 or physical <0:
    print("Invalid quantity")
else:
    expected = opening + received - dispatched
    difference = physical - expected
    print("Expected closing stock:", expected)

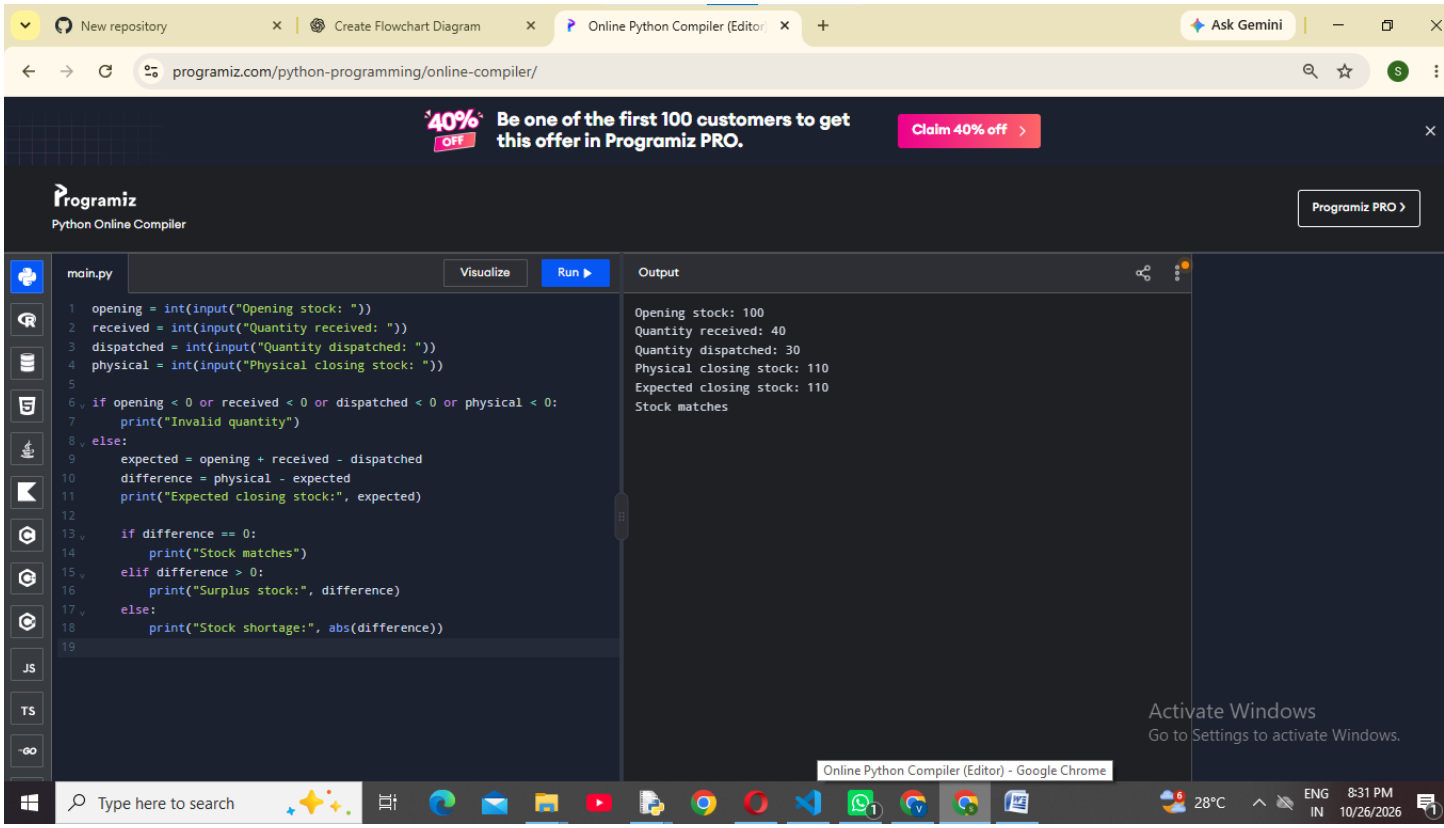
    if difference == 0:
        print("Stockmatches")
    elifdifference>0:
        print("Surplusstock:",difference)
```

```
else:
    print("Stockshortage:",abs(difference))
```

8. Test Cases and Sample Output

Test input	Expected output
100, 40, 30, 110	Expected closing stock: 110; Stock matches
100, 40, 30, 120	Expected closing stock: 110; Surplus stock: 10
100, 40, 30, 105	Expected closing stock: 110; Stock shortage: 5

Sample console output:.



9. Conclusion

The program reconciles stock records and reports whether the physical quantity matches the expected closing quantity.

Case Study 5: Student Timetable Conflict Detection

1. Problem Statement

Write a program to check whether two classes scheduled in the same room overlap in time.

2. Objectives

• Represent class details using tuples.

• Compare time intervals.

• Identify a room scheduling conflict.

3. Assumptions

• Times are represented as whole numbers using a 24-hour clock converted to minutes after midnight.

• A class interval includes its start and ends at its end time; back-to-back classes are allowed.

• The classes are in the same room.

4. Algorithm

1. Start.

2. Read start and end times for two classes in minutes after midnight.

3. Validate that each start is less than its end.

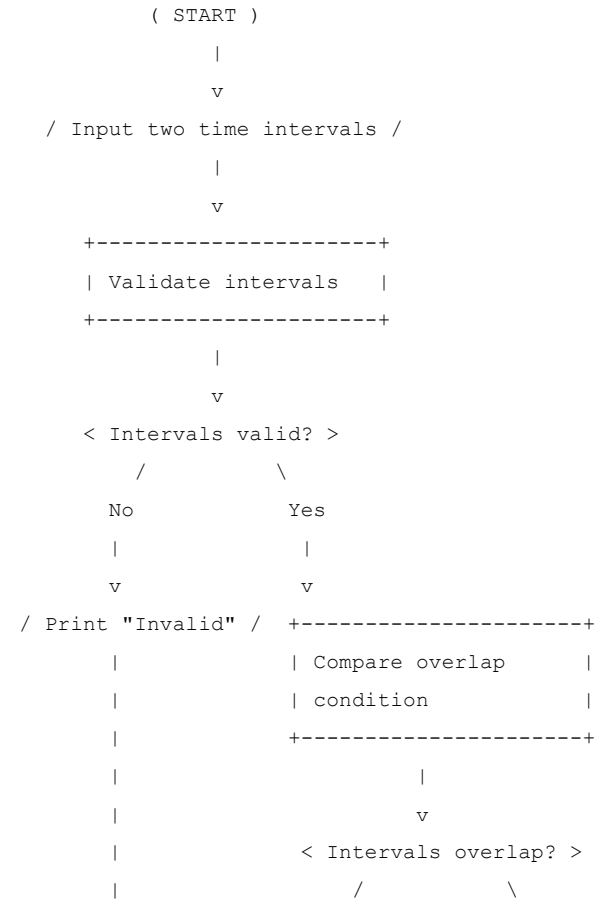
4. Represent each class as a tuple.

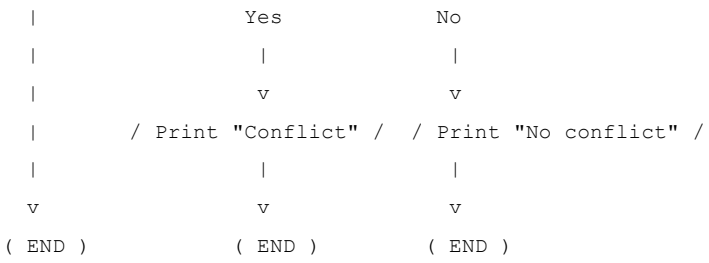
5. If first start is less than second end and second start is less than first end, report conflict.

6. Otherwise report no conflict.

7. Stop.

5. Flowchart





6. Pseudocode

```
BEGIN
  INPUTstart1,end1,start2,end2
  IF start1 >= end1 OR start2 >= end2
      PRINT "Invalid time interval"
  ELSE IF start1 < end2 AND start2 <end1
      PRINT "Timetable conflict"
  ELSE
      PRINT "No timetable conflict"
  END IF
END
```

7. Python Source Code

```
start1 = int(input("Class 1 start time (minutes after midnight): "))
end1 = int(input("Class 1 end time: "))
start2 = int(input("Class 2 start time: "))
end2 = int(input("Class 2 end time: "))

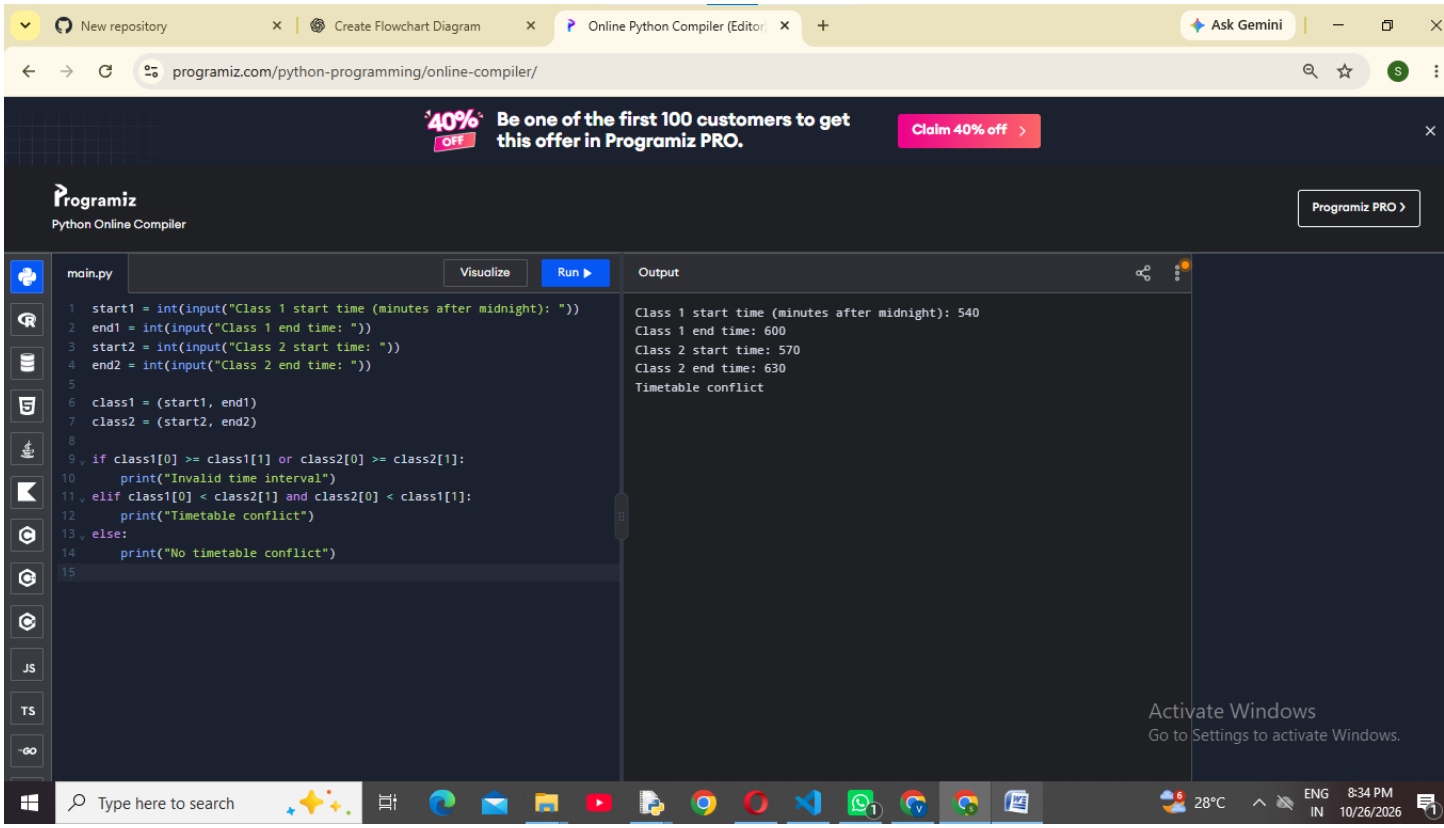
class1=(start1,end1)
class2=(start2,end2)

if class1[0] >= class1[1] or class2[0] >= class2[1]:
    print("Invalid time interval")
elif class1[0] < class2[1] and class2[0] <class1[1]:
    print("Timetable conflict")
else:
    print("Notimetableconflict")
```

8. Test Cases and Sample Output

Test input	Expected output
540, 600, 570, 630	Timetable conflict
540, 600, 600, 660	No timetable conflict
600, 540, 620, 680	Invalid time interval

Sampleconsoleoutput:



9. Conclusion

The program stores time intervals as tuples and uses an overlap condition to detect timetable conflicts.

Case Study 6: Repeated Transaction Detection

1. Problem Statement

Write a program to identify duplicate transaction IDs in a list of transactions.

2. Objectives

• Store transaction identifiers in a list.

• Use loops and conditions to detect repeated IDs.

• Display duplicate identifiers.

3. Assumptions

• Transaction IDs are entered as text separated by spaces.

• An ID is considered a duplicate if it appears more than once.

• The program reports each duplicate ID only once.

4. Algorithm

1. Start.

2. Read transaction IDs and split them into a list.

3. Create an empty list for duplicates.

4. Check each ID against the remaining IDs.

5. If it appears again and is not already recorded, add it to duplicates.

6. Display duplicate IDs or report none.

7. Stop.

5. Flowchart

```
START →Input transaction IDs →Create duplicate l
 each ID →Duplicates found? →Print result →END
```

6. Pseudocode

```
BEGIN
  INPUTlineofIDs
  ids=splitlineintolist
  duplicates = empty list
  FOR each id in ids
      IF id appears more than once AND id not in duplicates
          ADD id to duplicates
      ENDIF
  END FOR
  IF duplicates is empty: PRINT "No duplicates"
  ELSE: PRINT duplicates
END
```

7. Python Source Code

```
ids = input("Enter transaction IDs separated by spaces: ").split()
duplicates = []

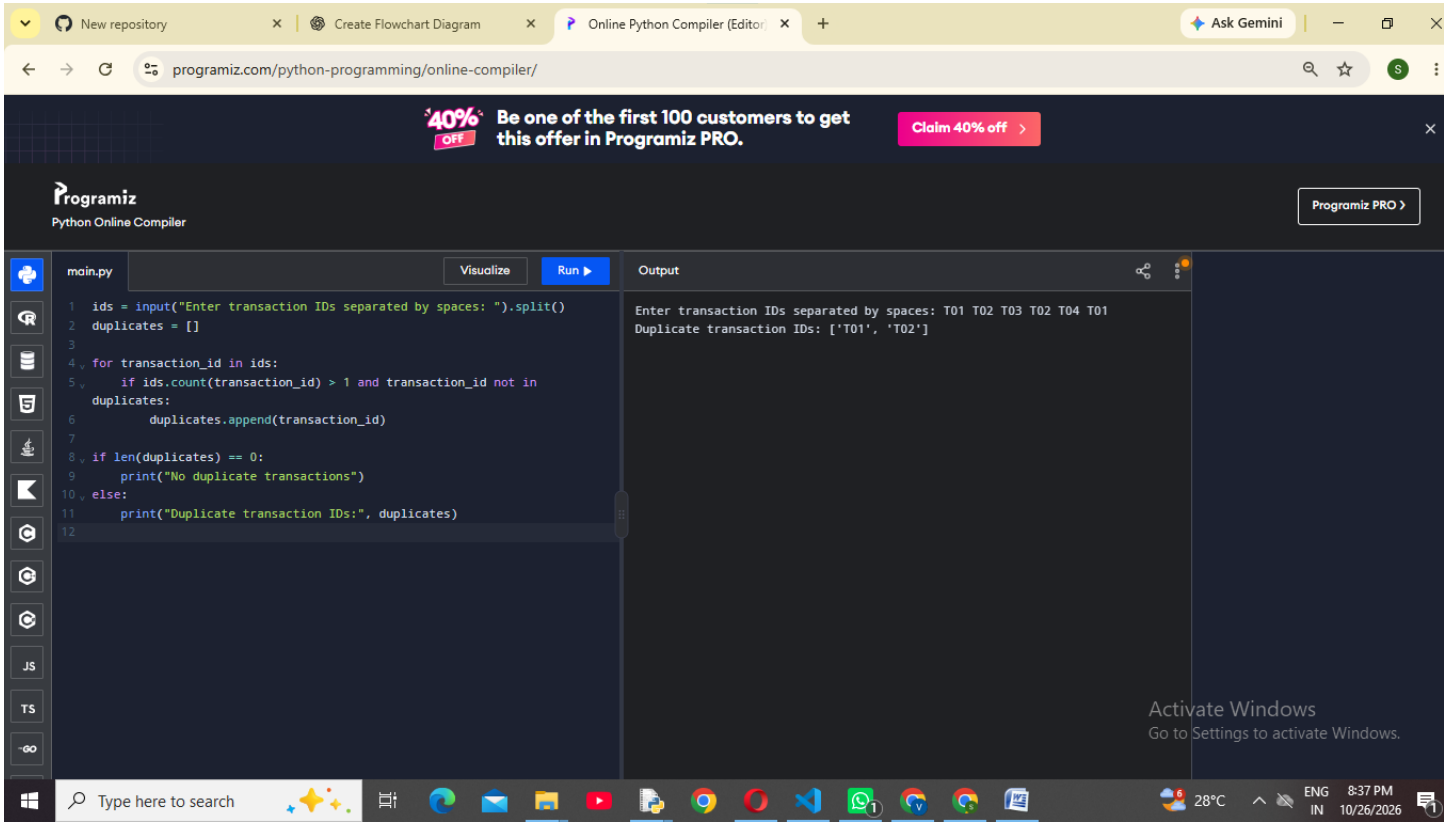
fortransaction_idinids:
    if ids.count(transaction_id) > 1 and transaction_id not in duplicates:
        duplicates.append(transaction_id)

iflen(duplicates)==0:
    print("No duplicate transactions")
else:
    print("DuplicatetransactionIDs:",duplicates)
```

8. Test Cases and Sample Output

Test input	Expected output
T01 T02 T03 T02 T04 T01	Duplicate transaction IDs: ['T02', 'T01']
A1 B2 C3	No duplicate transactions
X X X	Duplicate transaction IDs: ['X']

Sampleconsoleoutput:.



9. Conclusion

Theprogramuseslistoperationsandloopstofindrepeatedtransactionidentifierswithoutlistingthesameduplicate multiple times.

Case Study 7: Student Marks Matrix Analysis

1. Problem Statement

Writeaprogramtostoremarksformultiplestudentsinmultiplesubjectsandcalculateeachstudent'stotaland average.

2. Objectives

• Represent data using a nested list.

• Use nested loops to process marks.

• Calculate row-wise totals and averages.

3. Assumptions

• There are three students and three subjects.

• All marks are integers between 0 and 100.

• The sample matrix is provided directly in the program.

4. Algorithm

1. Start.

2. Create a nested list containing marks for three students.

3. For each student row, calculate the sum and average.

4. Display each student's marks, total, and average.

5. Stop.

5. Flowchart

```
START →Create marks matrix →Loop through each student row →Calculate total and average →Display results →
```

6. Pseudocode

```
BEGIN
  marks=[[80,75,90],[60,70,65],[95,88,92]]
  FOReachstudentrow
      total=sum(row)
      average = total / number of subjects
      PRINT row, total, average
  ENDFOR
END
```

7. Python Source Code

```
marks=[
    [80,75,90],
    [60,70,65],
    [95,88,92]
]

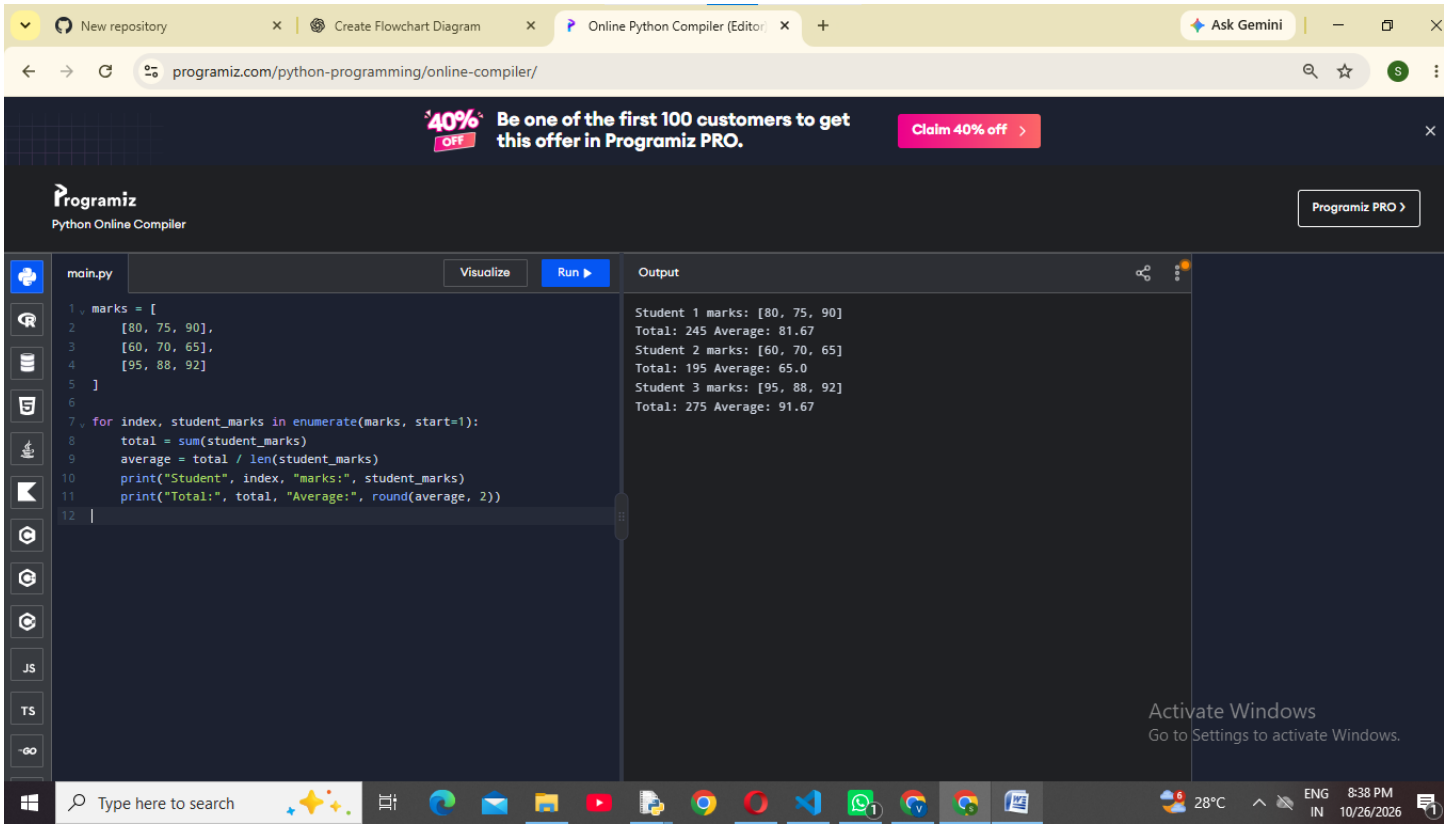
for index, student_marks in enumerate(marks, start=1):
    total = sum(student_marks)
    average = total / len(student_marks)
    print("Student", index, "marks:", student_marks)
    print("Total:",total,"Average:",round(average,2))
```

8. Test Cases and Sample Output

Student	Input marks	Expected output
1	[80, 75, 90]	Total: 245; Average: 81.67
2	[60, 70, 65]	Total: 195; Average: 65.0

Student	Input marks	Expected output
3	[95, 88, 92]	Total: 275; Average: 91.67

Sample console output:



9. Conclusion

The program demonstrates nested lists and loops to calculate student-wise totals and averages.

Case Study 8: Employee Shift Conflict Detection

1. Problem Statement

Write a program to check whether an employee has two shifts that overlap.

2. Objectives

• Store shift intervals as tuples.

• Compare two shifts using conditions.

• Report whether the shifts overlap.

3. Assumptions

• Shift times are integer minutes after midnight.

• Each shift starts before it ends.

• Two shifts overlap if each begins before the other ends.

4. Algorithm

1. Start.

2. Read start and end times for two shifts.

3. Store each shift in a tuple.

4. Validate both intervals.

5. Apply the overlap condition.

6. Display conflict or no conflict.

7. Stop.

5. Flowchart

```
START →Input shift times →Validate intervals →Check overlap →Print result →END
```

6. Pseudocode

```
BEGIN
  INPUT start1, end1, start2, end2
  shift1 = (start1, end1)
  shift2=(start2,end2)
  IF start1 >= end1 OR start2 >= end2
      PRINT "Invalid shift"
  ELSE IF start1 < end2 AND start2 <end1
      PRINT "Shift conflict"
  ELSE
      PRINT"Noshiftconflict" END
  IF
END
```

7. Python Source Code

```
start1 = int(input("First shift start (minutes after midnight): "))
end1 = int(input("First shift end: "))
start2 = int(input("Second shift start: "))
end2 = int(input("Second shift end: "))

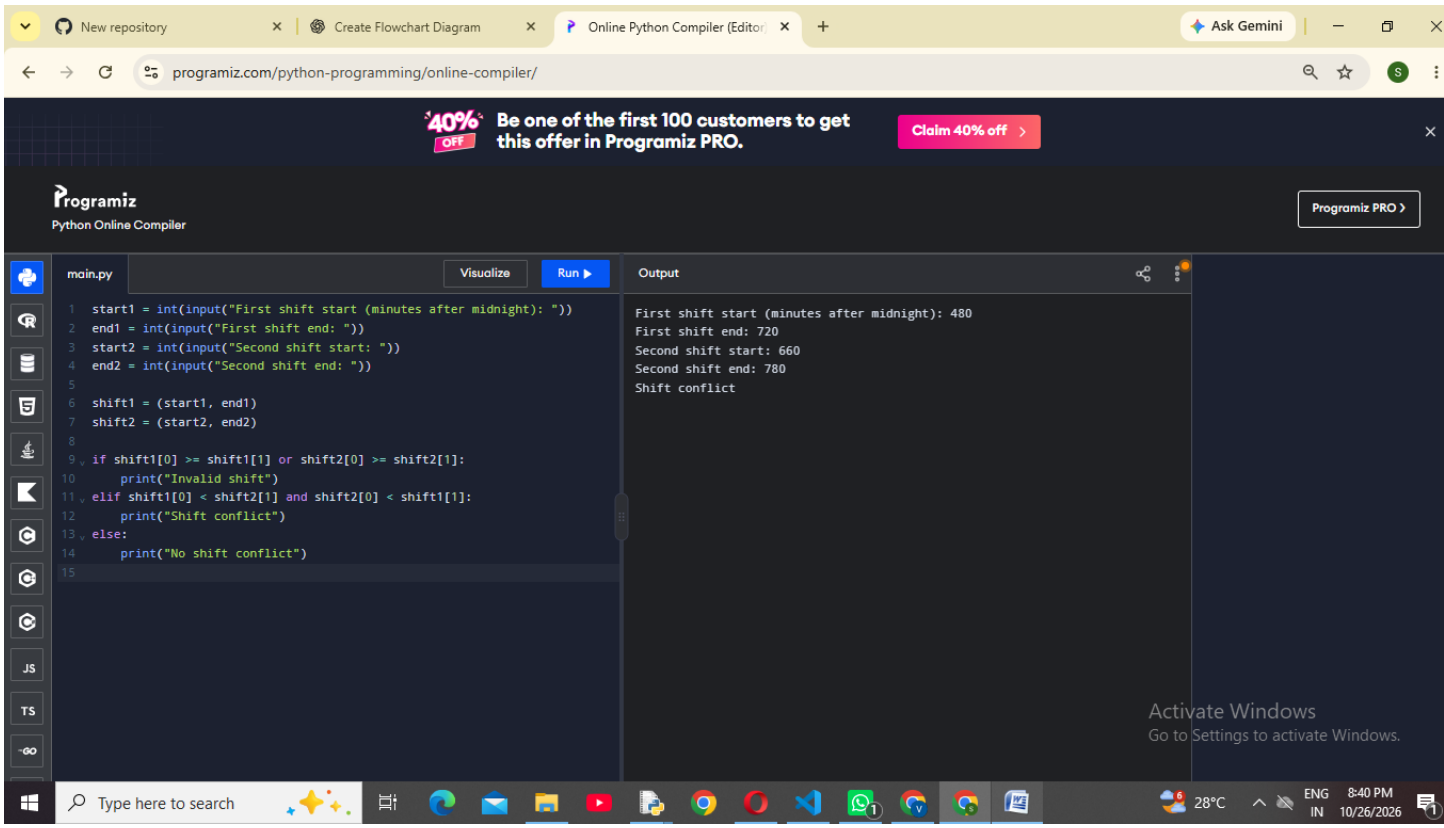
shift1=(start1,end1)
shift2=(start2,end2)

if shift1[0] >= shift1[1] or shift2[0] >= shift2[1]:
    print("Invalid shift")
elif shift1[0] < shift2[1] and shift2[0] <shift1[1]:
    print("Shift conflict")
else:
    print("Noshiftconflict")
```

8. Test Cases and Sample Output

Test input	Expected output
480, 720, 660, 780	Shift conflict
480, 600, 600, 720	No shift conflict
700, 650, 800, 900	Invalid shift

Sample console output :



9. Conclusion

The program compares tuple-based time intervals and detects overlapping employee shifts.

Case Study 9: Student Academic Risk Classification

1. Problem Statement

Write a program to classify a student's academic risk using attendance and average marks.

2. Objectives

• Use multiple conditions.

• Classify students into simple risk categories.

• Display the reason for the classification.

3. Assumptions

• Attendance and average marks are percentages from 0 to 100.

• High risk: attendance below 60 or average marks below 40.

• Medium risk: attendance below 75 or average marks below 60, if not high risk.

• Otherwise, risk is low.

4. Algorithm

1. Start.

2. Read attendance and average marks.

3. Validate both values are between 0 and 100.

4. If attendance is below 60 or marks below 40, classify as high risk.

5. Else if attendance is below 75 or marks below 60, classify as medium risk.

6. Otherwise classify as low risk.

7. Display category.

8. Stop.

5. Flowchart

```
START →Input attendance and average →Validate →High-risk rule? →Medium-risk rule? →Set category →Print
```

6. Pseudocode

```
BEGIN
  INPUTattendance,average
  IF either value is outside 0..100
      PRINT "Invalid input"
  ELSE IF attendance < 60 OR average <40
      PRINT "High risk"
  ELSE IF attendance < 75 OR average <60
      PRINT "Medium risk"
  ELSE
      PRINT"Lowrisk" END
  IF
END
```

7. Python Source Code

```
attendance = float(input("Enter attendance (%): "))
average=float(input("Enteraveragemarks(%):"))

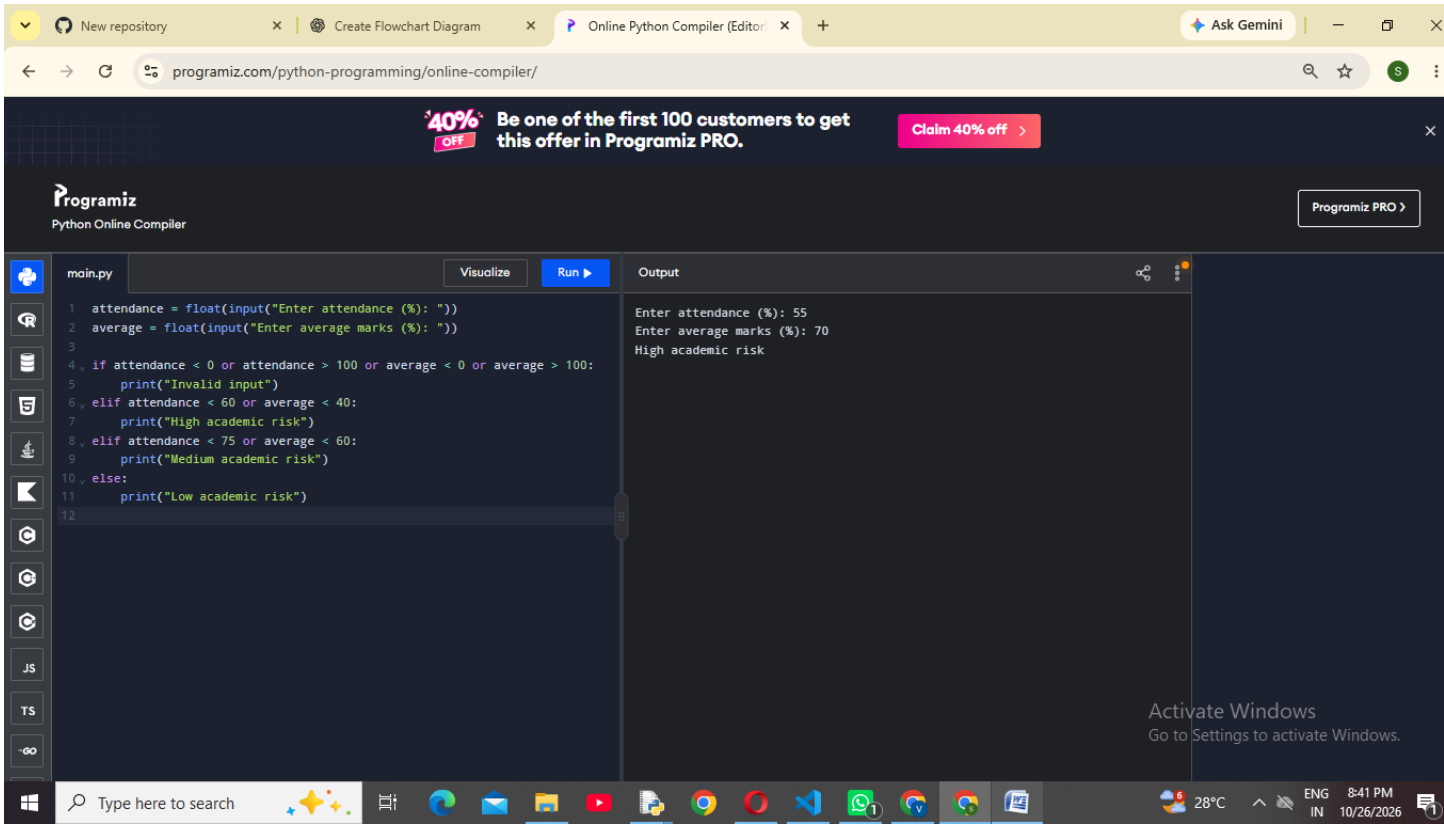
if attendance < 0 or attendance > 100 or average < 0 or average >100:
    print("Invalid input")
elif attendance < 60 or average <40:
    print("High academic risk")
elif attendance < 75 or average <60:
    print("Medium academic risk")
else:
```

```
print("Lowacademicrisk")
```

8. Test Cases and Sample Output

Test input	Expected output
55, 70	High academic risk
70, 65	Medium academic risk
85, 80	Low academic risk

Sampleconsoleoutput:



9. Conclusion

The program combines attendance and marks using conditional statements to classify a student's academic risk.

Case Study 10: Multi-Branch Retail Inventory Transfer

1. Problem Statement

Writeaprogramtocomparestockintwobranchesagainsttargetquantitiesandcalculateasimpletransferfroma branch with surplus to a branch with shortage.

2. Objectives

• Use lists to store stock quantities.

• Calculate surplus and shortage.

• Determine a possible transfer quantity.

3. Assumptions

• The example handles one product in two branches.

• Stock quantities and target quantities are non-negative integers.

• A transfer is possible only up to the available surplus and the shortage.

4. Algorithm

1. Start.

2. Read Branch A stock, Branch B stock, and each branch's target stock.

3. Validate all quantities.

4. Calculate Branch A surplus and Branch B shortage.

5. Transfer the smaller of surplus and shortage if both are positive.

6. Display transfer quantity and updated stock.

7. Stop.

5. Flowchart

```
START →Input stocks and targets →Validate →Calculate surplus/shortage →Transfer min(surplus, shortage) →
```

6. Pseudocode

```
BEGIN
  INPUTstockA,targetA,stockB,targetB
  IF any value < 0: PRINT "Invalid
  quantity"ELSE
      surplusA = max(0, stockA - targetA)
      shortageB = max(0, targetB - stockB)
      transfer = min(surplusA, shortageB)
      stockA = stockA - transfer
      stockB = stockB + transfer
      PRINT transfer, stockA, stockB
  ENDIF
END
```

7. Python Source Code

```
stock_a = int(input("Branch A current stock: "))
target_a = int(input("Branch A target stock: "))
stock_b = int(input("Branch B current stock: "))
target_b=int(input("BranchBtargetstock:"))

if min(stock_a, target_a, stock_b, target_b) <0:
    print("Invalid quantity")
else:
    surplus_a = max(0, stock_a - target_a)
    shortage_b = max(0, target_b - stock_b)
    transfer = min(surplus_a, shortage_b)
```

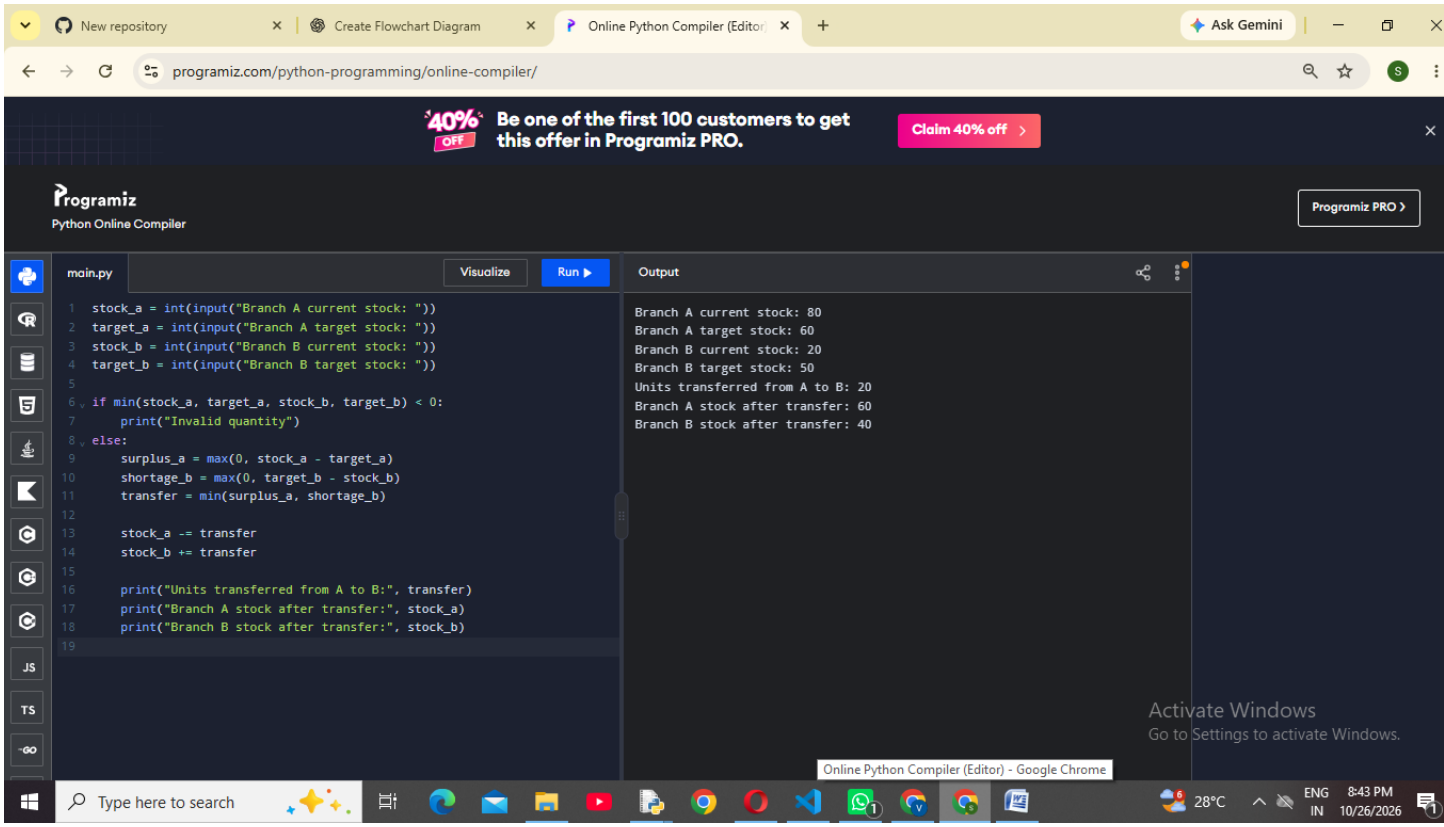
```
stock_a-=transfer
stock_b+=transfer

print("Units transferred from A to B:", transfer)
print("Branch A stock after transfer:", stock_a)
print("Branch B stock after transfer:", stock_b)
```

8. Test Cases and Sample Output

Test input	Expected output
80, 60, 20, 50	Transferred: 20; Branch A: 60; Branch B: 40
70, 60, 20, 50	Transferred: 10; Branch A: 60; Branch B: 30
50, 60, 20, 50	Transferred: 0; Branch A: 50; Branch B: 20

Sampleconsoleoutput:



.

9. Conclusion

The program calculates the available surplus and shortage, then transfers only the quantity that can be supplied.